

YouTube Backend API

Complete endpoint reference for a production-style video platform backend — authentication, videos, comments, playlists, subscriptions, tweets, likes, and dashboard analytics.

- NODE.JS
- EXPRESS.JS
- MONGODB
- MONGOOSE
- JWT AUTH
- CLOUDINARY
- MULTER
- BCRYPT
- POSTMAN TESTED

Contents

01	Authentication	Overview
02	Users	11 endpoints
03	Videos	6 endpoints
04	Comments	4 endpoints
05	Playlists	7 endpoints
06	Subscriptions	3 endpoints
07	Tweets	4 endpoints
08	Likes	4 endpoints
09	Dashboard	2 endpoints
10	API Testing	Postman
11	Technology	Stack overview

Request conventions — JSON endpoints use `application/json`. File-upload endpoints use `multipart/form-data`. Route parameters such as `:videoId` and `:userId` refer to MongoDB document IDs. Query parameters are optional unless otherwise noted.

01

Authentication

JWT + HTTP-only cookies

Protected endpoints require a valid JWT access token, supplied via the `Authorization` header. The API also accepts HTTP-only cookies for both access and refresh tokens, so the same routes work seamlessly for browser-based and token-based clients.

Authorization header

```
Authorization: Bearer <access_token>
```

Typical authenticated request

```
GET http://localhost:8000/api/v1/users/current/user
Authorization: Bearer <access_token>
```

POST

/users/register

NOT REQUIRED

Creates a new user account. Content-Type: multipart/form-data

FIELD	TYPE	REQUIRED
fullName	Text	Yes
email	Text	Yes
username	Text	Yes
password	Text	Yes
avatar	File	Yes
coverImage	File	No

POST

/users/login

NOT REQUIRED

Authenticates a user. Content-Type: application/json

```
{
  "email": "user@gmail.com",
  "password": "12345678"
}
```

POST

/users/logout

REQUIRED

Logs out the authenticated user and clears auth cookies.

POST

/users/refresh-token

NOT REQUIRED

Uses a valid refresh token to issue a new access token.

POST

/users/change-password

REQUIRED

Changes the current user's password. Content-Type: application/json

```
{
  "oldPassword": "oldpassword",
  "newPassword": "newpassword"
}
```

GET

/users/current/user

REQUIRED

Returns the authenticated user's information.

PATCH

/users/update-account

REQUIRED

Updates account details. Content-Type: application/json

```
{
  "fullName": "Updated Name",
  "email": "updated@gmail.com"
}
```

PATCH

/users/avatar

REQUIRED

Updates the user's avatar. Content-Type: multipart/form-data

FIELD	TYPE	REQUIRED
avatar	File	Yes

PATCH

/users/cover-image

REQUIRED

Updates the user's cover image. Content-Type: multipart/form-data

FIELD	TYPE	REQUIRED
coverImage	File	Yes

GET

/users/c/:username

REQUIRED

Returns the requested user's channel profile.

```
GET http://localhost:8000/api/v1/users/c/hit
```

GET

/users/history

REQUIRED

Returns the authenticated user's watch history.

GET

/videos

REQUIRED

Returns published videos with search and pagination options.

QUERY PARAMETER	DESCRIPTION
page	Page number
limit	Number of videos per page
query	Search videos by title
userId	Filter videos by user

```
GET http://localhost:8000/api/v1/videos?page=1&limit=10
```

```
GET http://localhost:8000/api/v1/videos?query=node
```

POST

/videos

REQUIRED

Uploads a video and thumbnail and creates the video record. Content-Type: multipart/form-data

FIELD	TYPE	REQUIRED
title	Text	Yes
description	Text	Yes
videoFile	File	Yes
thumbnail	File	Yes

Video and thumbnail files are uploaded to Cloudinary.

GET

/videos/:videoId

REQUIRED

Returns a video by its ID.

```
GET http://localhost:8000/api/v1/videos/VIDEO_ID
```

PATCH

/videos/:videoId

REQUIRED

Updates video details and accepts a thumbnail upload. Content-Type: multipart/form-data

```
{
  "title": "Updated Video Title",
  "description": "Updated video description"
}
```

DELETE

/videos/:videoId

REQUIRED

Deletes the video record from MongoDB and removes the associated video and thumbnail assets from Cloudinary.

PATCH

/videos/toggle/publish/:videoId

REQUIRED

Toggles the video's isPublished status.

GET

/comments/:videoId

REQUIRED

Returns comments for a video with pagination.

QUERY PARAMETER	DESCRIPTION
page	Page number
limit	Number of comments per page

```
GET http://localhost:8000/api/v1/comments/VIDEO_ID?page=1&limit=10
```

POST

/comments/:videoId

REQUIRED

Adds a comment to a video. Content-Type: application/json

```
{ "content": "Great video!" }
```

PATCH

/comments/c/:commentId

REQUIRED

Updates a comment. Content-Type: application/json

```
{ "content": "Updated comment" }
```

DELETE

/comments/c/:commentId

REQUIRED

Deletes a comment.

POST

/playlist

REQUIRED

Creates a playlist. Content-Type: application/json

```
{
  "name": "My Favorites",
  "description": "My favorite videos"
}
```

GET

/playlist/:playlistId

REQUIRED

Returns a playlist by ID.

GET

/playlist/user/:userId

REQUIRED

Returns playlists for a user.

PATCH

/playlist/:playlistId

REQUIRED

Updates playlist details. Content-Type: application/json

```
{
  "name": "Updated Playlist",
  "description": "Updated description"
}
```

DELETE

/playlist/:playlistId

REQUIRED

Deletes a playlist.

PATCH

/playlist/add/:videoId/:playlistId

REQUIRED

Adds a video to a playlist.

PATCH

/playlist/remove/:videoId/:playlistId

REQUIRED

Removes a video from a playlist.

POST

/subscriptions/c/:channelId

REQUIRED

Toggles the authenticated user's subscription to a channel — creates a subscription if none exists, or removes it if one does. No request body required.

```
POST http://localhost:8000/api/v1/subscriptions/c/CHANNEL_ID
```

GET

/subscriptions/c/:channelId

REQUIRED

Returns users subscribed to the specified channel.

GET

/subscriptions/u/:subscriberId

REQUIRED

Returns channels subscribed to by the specified user.

07

Tweets

4 endpoints

POST

/tweets

REQUIRED

Creates a tweet. Content-Type: application/json

```
{ "content": "Hello from my backend!" }
```

GET

/tweets/user/:userId

REQUIRED

Returns tweets for a user.

PATCH

/tweets/:tweetId

REQUIRED

Updates a tweet. Content-Type: application/json

```
{ "content": "Updated tweet" }
```

DELETE

/tweets/:tweetId

REQUIRED

Deletes a tweet.

POST

`/likes/toggle/v/:videoId`

REQUIRED

Toggles the like status of a video.

POST

`/likes/toggle/c/:commentId`

REQUIRED

Toggles the like status of a comment.

POST

`/likes/toggle/t/:tweetId`

REQUIRED

Toggles the like status of a tweet.

GET

`/likes/videos`

REQUIRED

Returns videos liked by the authenticated user.

GET

/dashboard/stats

REQUIRED

Returns channel statistics for the authenticated user.

GET

/dashboard/videos

REQUIRED

Returns videos associated with the authenticated user's channel.

Every endpoint in this reference has been manually tested end-to-end using Postman. Coverage includes:

[User auth & profile](#)[Video management](#)[Comments](#)[Playlists](#)[Subscriptions](#)[Tweets](#)[Likes](#)[Dashboard](#)

TECHNOLOGY	PURPOSE
Node.js	Backend JavaScript runtime
Express.js	API framework and routing
MongoDB	Primary database
Mongoose	MongoDB object modeling (ODM)
JWT	Access & refresh token authentication
Cloudinary	Video and image storage / delivery
Multer	Multipart file upload handling
bcrypt	Password hashing
Postman	API testing